

附录 C：实验四深度分割指南

实验配置

- 下载压缩包
<https://bhpan.buaa.edu.cn/link/AAB14032EEC767484AA750C7C2677DA659>
文件夹包含：detectron2-main.zip,shutu_rcnn.tar,weights.zip
图像分割任务及标注：val2017.zip, annotations_trainval2017.zip
- 将 detectron2 和 weights 解压，shutu_rcnn 不解压

运行环境（配置好的 docker）

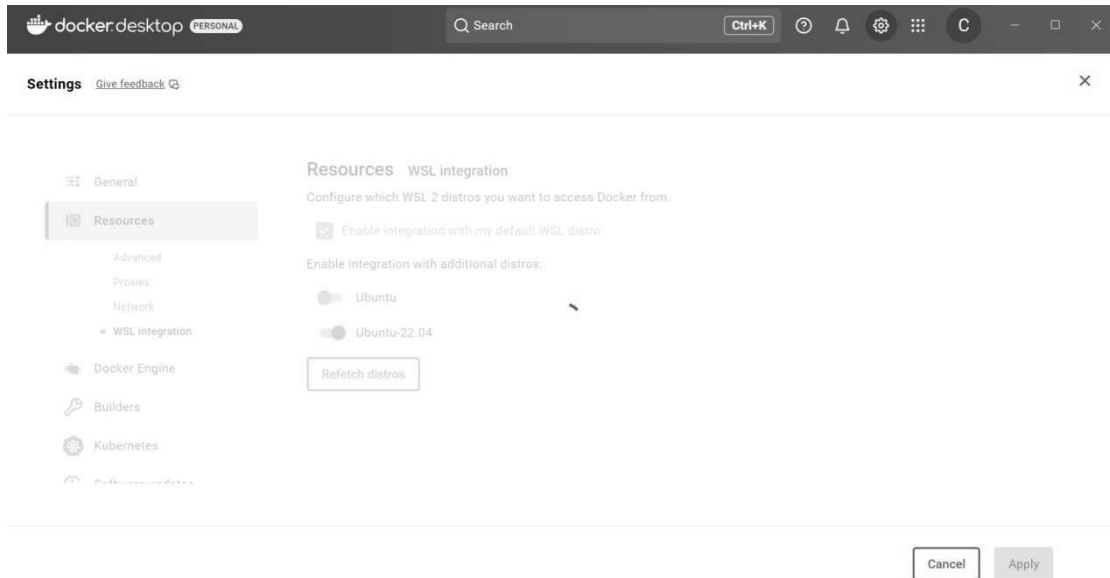
- 安装 WSL
线上方式（如遇操作超时多试几次就好，安装完之后重启）
电脑上按 win+R，输入 cmd

```
1. # 在命令行 cmd 中运行
2. wsl --install #重启
3. wsl --install -d Ubuntu-22.04
4. wsl --set-default-version 2
5. wsl --set-default Ubuntu-22.04
```

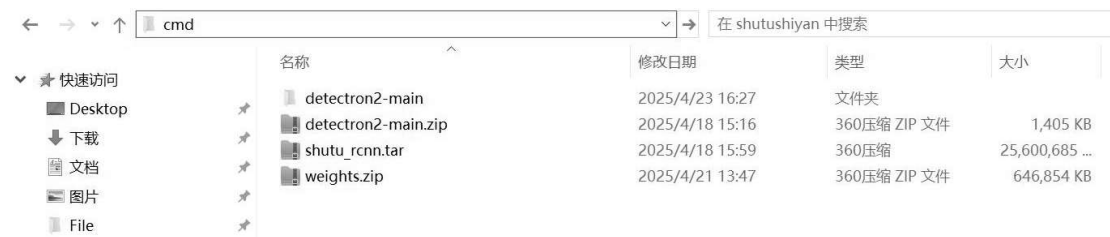
- 配置 NVIDIA Container Toolkit 英伟达驱动，[教程]

```
1. # 进入 WSL-2
2. wsl -d Ubuntu-22.04
3. # 1.添加库（右键粘贴）
4. curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg \
5. && curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
6. sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
7. sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
8.
9. # 2.更新 apt
10. sudo apt-get update
11.
12. # 3.安装
13. sudo apt-get install -y nvidia-container-toolkit
```

- 安装 docker
 - 运行[Docker Desktop Installer]，直接在 windows 上进行安装
 - 在设置界面允许 WSL 使用



- 提前下载 shutu_rcnn.tar
- docker 加载镜像 (使用 cmd 运行, 按如下方式在存有压缩包的文件夹中输入并回车打开 cmd)
- detectron2-main 的绝对目录不能有中文, 建议新建文件夹



cmd 中运行:

1. `wsl -d Ubuntu-22.04`
2. # 注意后续的运行都在 wsl 中, 也就是需要首先 `wsl -d Ubuntu-22.04`
3. # 加载 docker image
4. `docker load -i shutu_rcnn.tar`
5. # docker 容器, 在容器的命令行之中完成实验, 注意 detectron2-main 的绝对目录不能有中文
6. `docker run -it --name mask_rcnn -v /mnt/e/detectron2-main:/mnt/data --gpus all shutu_rcnn /bin/bash`
7. # 上面命令的 detectron2-main 在/mnt/XXX(CDE 盘)/XXX 目录下, 自行修改 /mnt 之后的路径为自己 cmd 上显示的路径
8. #例
如: `docker run -it --name mask_rcnn -v /mnt/e/shutushiyen/detectron2-main:/mnt/data --gpus all shutu_rcnn /bin/bash`
9. #进入容器之后
10. `cd /mnt/data`
11. `conda activate rcnn`
- 12.
13. #部分 docker 指令:

```
14. # 退出容器
15. 快捷键: Ctrl+P 之后 Ctrl+Q
16. # 终止容器
17. exit (Ctrl+D)
18. # 启动容器
19. docker start mask_rcnn
20. # 进入容器
21. docker attach mask_rcnn
```

备注：后续的训练、测试环境都在 mask_rcnn 容器中进行

使用预训练模型进行分割

● 运行指令

```
1. cd /mnt/data
2. conda activate rcnn
3. cd demo
4. mkdir results
5. # GPU 运行
6. python demo.py --config-file ../configs/COCO-InstanceSegmentation/mask_rcnn
_R_50_FPN_3x.yaml --input ../datasets/coco/val2017/000000000724.jpg --output resul
ts --opts MODEL.WEIGHTS ../weights/R50_FPN_3x/model_final_f10217.pkl
7. # CPU 运行（无显卡）
8. python demo.py --config-file ../configs/COCO-InstanceSegmentation/mask_rcnn
_R_50_FPN_3x.yaml --input ../datasets/coco/val2017/000000000724.jpg --output resul
ts --opts MODEL.WEIGHTS ../weights/R50_FPN_3x/model_final_f10217.pkl MODEL.DEVICE
cpu
```

● 得到输出的分割图像在 demo/results 文件夹里面